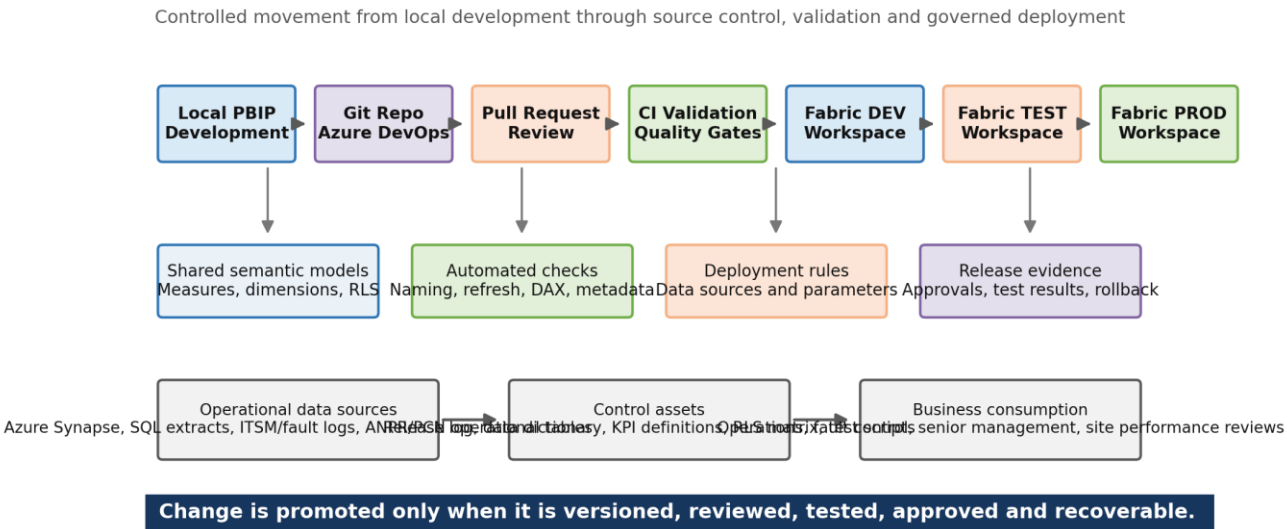


Power BI CI/CD and Version Control Framework

Analytics, Fault Control and Operational Reporting

Power BI CI/CD Target Architecture



Document type	Power BI CI/CD governance and implementation framework
Business area	Analytics, Fault Control, operational reporting and performance monitoring
Version	1.0
Date	May 2026
Owner	Teruyuki Ito

Document Control

Version	Date	Change summary	Owner / reviewer
0.1	June 2026	Initial working draft covering Power BI source control, release gates and deployment model.	Analytics & Fault Control
0.9	June 2026	Validated against operational reporting use cases and agreed workspace promotion controls.	Analytics & Fault Control / Operations
1.0	June 2026	Baseline version for controlled adoption across Power BI dashboards and semantic models.	Analytics & Fault Control

Table of Contents

- 1. Purpose and Operating Context
- 2. Framework Objectives
- 3. Current Challenges Addressed
- 4. Target Power BI CI/CD Architecture
- 5. Repository and Version Control Standards
- 6. Branching Strategy
- 7. Environment and Workspace Model
- 8. Development and Deployment Process
- 9. Release Gates and Quality Controls
- 10. Testing Framework
- 11. Deployment Rules and Configuration Management
- 12. Roles and Responsibilities
- 13. Naming Standards and Folder Structure
- 14. Release Calendar and Operational Cadence
- 15. Implementation Roadmap
- 16. Appendices

1. Purpose and Operating Context

This framework defines how Power BI reports, semantic models and related reporting assets are developed, reviewed, versioned, tested and released across the analytics environment. The intent is to move Power BI delivery away from isolated desktop files and unmanaged workspace edits, and towards a controlled release process with visible ownership, repeatable checks and auditable change history.

The framework was designed for an operational reporting environment where dashboards support daily decision-making across parking operations, fault control, site performance, ANPR camera monitoring, PCN funnel visibility, exemptions, processing exceptions and management reporting. In this context, small reporting changes can affect operational interpretation, stakeholder confidence and business actions; therefore each change must be traceable, validated and recoverable.

The approach combines Power BI Project files, Git version control, workspace separation, release evidence, approval gates and deployment controls. It is intentionally practical: it can be used by a small analytics function without creating unnecessary overhead, but it still provides enough discipline to support production reporting.

Design Principles

Principle	How it is applied
Controlled	Production dashboards are changed only through an approved release path.
Traceable	Each report, measure and model change has a commit, pull request or release note.
Repeatable	The same validation checklist is applied for similar dashboard and model releases.
Recoverable	Every production release is tagged so the previous stable version can be restored.
Business-aligned	Operational owners sign off KPI, definition and visual changes before production.
Secure	Workspace permissions, row-level security and data source credentials are controlled separately by environment.

2. Framework Objectives

- Introduce version control for Power BI dashboards and semantic models using a repository-based structure.
- Separate development, testing, UAT and production workspaces so that live reports are not changed directly.
- Create an agreed branching and release model covering normal changes, urgent hotfixes and production rollback.
- Define quality gates for DAX measures, model relationships, Power Query steps, visual layout, refresh performance, data reconciliation and row-level security.
- Standardise naming conventions, folder structure, commit messages and release documentation.
- Build a lightweight operating cadence so dashboard changes are visible to operations and senior stakeholders.

3. Current Challenges Addressed

Challenge	Operational risk	Framework response
Multiple versions of the same dashboard	Analysts may save local PBIX copies with similar names, making it difficult to identify the latest approved version.	Repository history and version tags identify the approved baseline.
Direct edits in production workspaces	Quick fixes can bypass testing, creating risk to operational reporting during daily reviews.	Promotion path prevents untested production changes.
Limited change evidence	Changes to KPI logic, filters, date rules or visuals may not be documented clearly enough for audit or handover.	Release notes and PR templates capture definition-level changes.
Unclear ownership	It can be difficult to confirm who owns each semantic model, dashboard, data refresh or business definition.	RACI matrix clarifies ownership across analytics, operations and workspace administration.
Manual deployment steps	Manual publishing creates variation between environments and increases	Deployment gates and workspace rules standardise movement between

Challenge	Operational risk	Framework response
	the risk of overwriting production content.	environments.
Weak rollback capability	Without release tags and retained deployment evidence, reverting to a previous stable version becomes slow and uncertain.	Tagged releases and rollback playbook restore previous approved artefacts.

4. Target Power BI CI/CD Architecture

The target architecture uses a controlled path from local report/model development to Git, then through automated validation and staged workspaces. The key change is that the Git repository becomes the reference point for approved artefacts rather than a shared folder or an individual's desktop file.

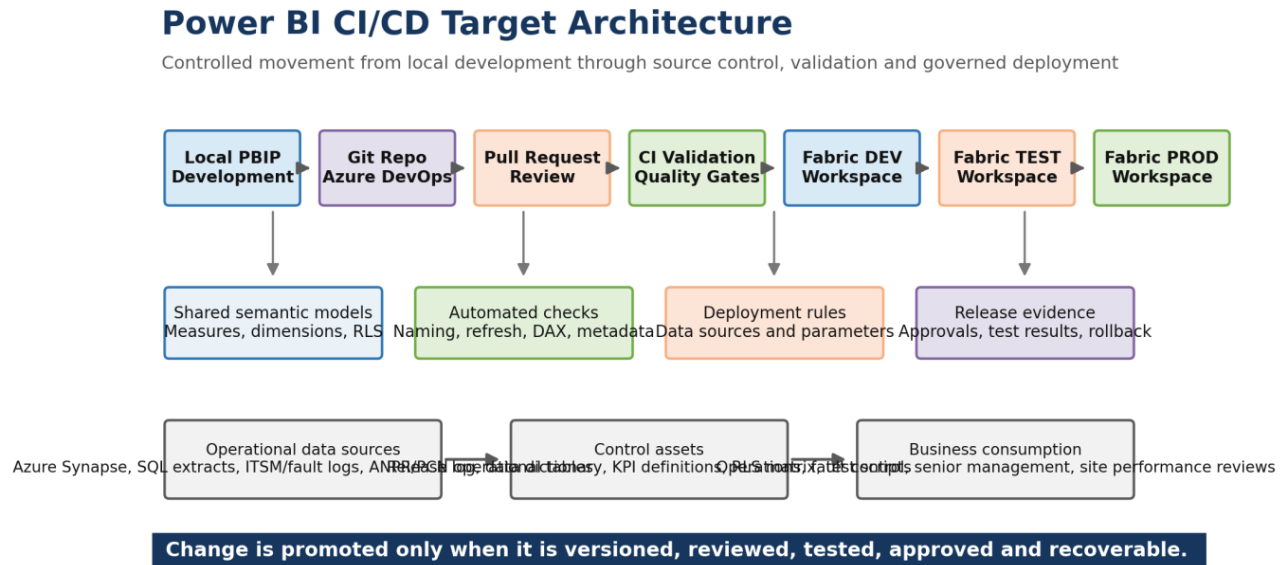


Figure 1. Target architecture for Power BI CI/CD, release control and operational dashboard promotion.

Architecture Components

Component	Purpose in the framework
Power BI Project / PBIP	Used for report and semantic model development where artefacts can be stored as text-based project files suitable for source control.
Git repository	Stores the controlled version of reports, semantic models, documentation, release notes and deployment configuration.
Pull request workflow	Provides peer review, change evidence and controlled merging into shared branches.
CI validation	Runs repeatable checks before deployment, including file structure, metadata, model quality and report standards.
Fabric / Power BI workspaces	Separate development, test/UAT and production workspaces reduce the risk of unapproved production changes.
Deployment rules	Manage environment-specific configuration such as data sources and parameters.
Release evidence pack	Captures deployment ID, version tag, test results, sign-off, known issues and rollback notes.

5. Repository and Version Control Standards

The repository should be structured so that an analyst, reviewer or support person can quickly identify the report, semantic model, deployment scripts and supporting documentation associated with each dashboard release.

```

/powerbi-cicd/
  /reports/
    /operations-performance/
      OperationsPerformance.Report/
      OperationsPerformance.SemanticModel/
      README.md
    /fault-control-monitoring/
      FaultControlMonitoring.Report/
      FaultControlMonitoring.SemanticModel/

```

```

    README.md
  /shared-semantic-models/
    /core-operations-model/
  /deployment/
    pipeline.yml
    deployment-rules-dev-test-prod.json
    workspace-map.json
  /tests/
    measure-validation/
    refresh-validation/
    rls-validation/
  /docs/
    data-dictionary/
    kpi-definitions/
    release-notes/
    architecture-decisions/
  CHANGELOG.md
  README.md

```

Standard	Requirement
Repository is the source of truth	The approved version of a report or model is the version committed and tagged in the repository, not an unpublished local copy.
No direct production overwrite	Reports are not published manually to production unless an emergency hotfix process is invoked and documented.
Every change has a work item	Each change should reference an issue, ticket or change request ID.
Small commits are preferred	Commits should separate data model, DAX, Power Query, visual and documentation changes where practical.
Release notes are mandatory	Every production deployment must include a short release note with impact, test evidence and rollback position.
Sensitive data is excluded	Data exports, credentials, secrets, gateway keys and personal data must not be committed to the repository.

6. Branching Strategy

The branching strategy balances control with speed. It is deliberately simple enough for a small analytics team but formal enough to protect production dashboards.

Branching and Version Control Model

Feature work is isolated, reviewed through pull requests and promoted through release branches before production tagging.

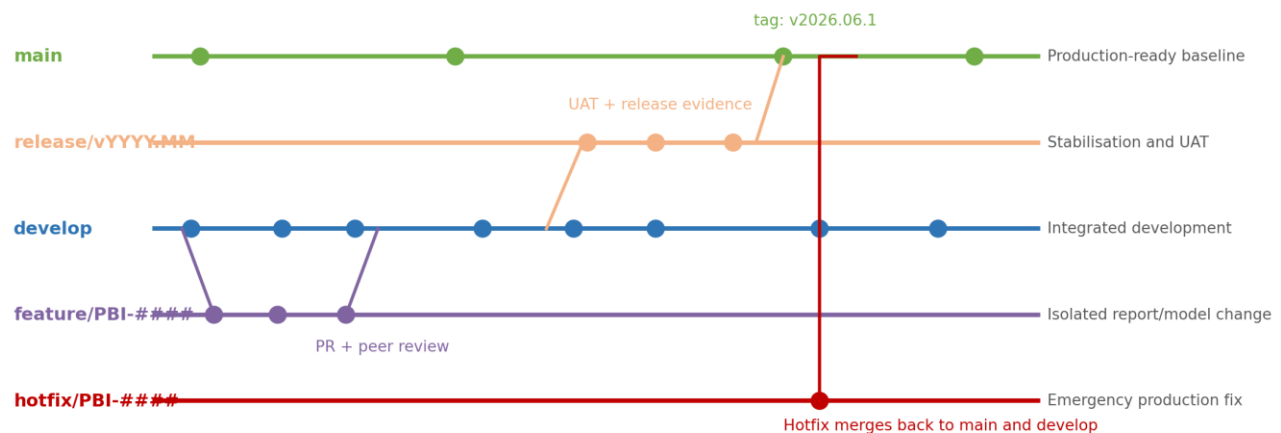


Figure 2. Branching model for feature changes, releases, hotfixes and production tagging.

Branch	Usage rule
main	Represents the production-approved baseline. Only release and hotfix branches are merged into main.
develop	Integration branch for completed changes that have passed

Branch	Usage rule
	peer review and are ready for DEV workspace deployment.
feature/PBI-####-short-description	Used for normal report, model, DAX, Power Query, RLS or documentation changes.
release/vYYYY.MM	Used to stabilise a planned production release, complete UAT and freeze scope.
hotfix/PBI-####-short-description	Used only for urgent production issues where waiting for the normal release cycle would create material operational risk.

7. Environment and Workspace Model

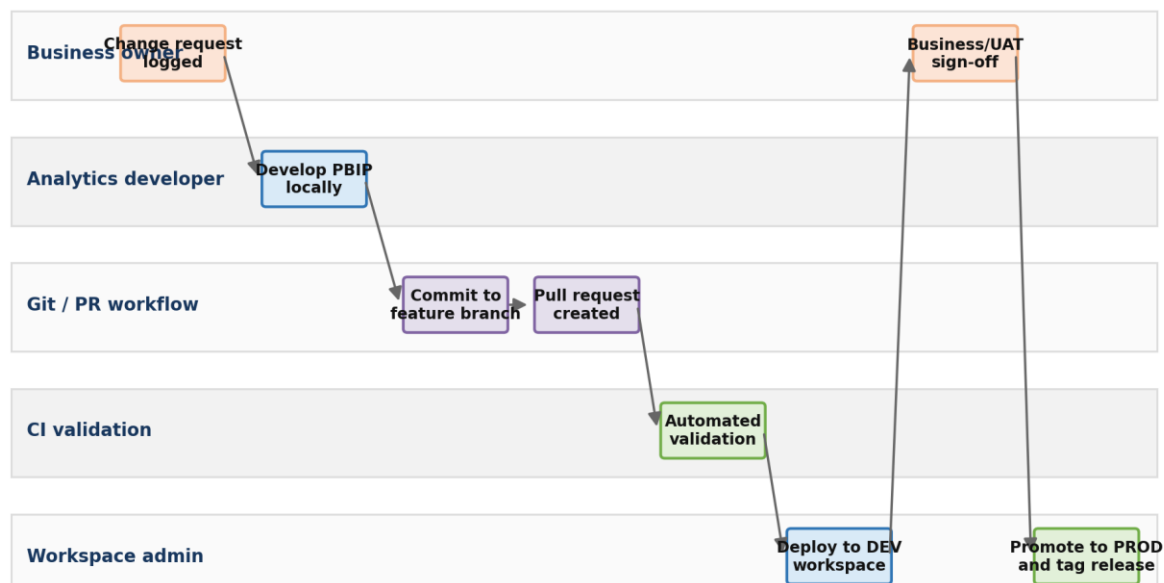
The workspace model separates experimentation from operational reporting. Each environment has a defined purpose, permission model and deployment rule set.

Environment	Location	Purpose	Access principle
Local	Analyst desktop / local repo	Build report/model changes, validate locally, commit to feature branch.	Developer only
DEV	Power BI/Fabric DEV workspace	Integrated development testing, early refresh checks and technical validation.	Analytics team
TEST	Power BI/Fabric TEST workspace	Regression testing, comparison to current production, refresh scheduling and data source validation.	Analytics + workspace admin
UAT	Power BI/Fabric UAT workspace or controlled app audience	Business validation of KPI definitions, layout, filters and operational interpretation.	Operations owners + analytics
PROD	Power BI/Fabric PROD workspace / app	Certified dashboard consumption for operations and management reporting.	Viewers only, limited publishers

8. Development and Deployment Process

The end-to-end process starts with a logged change and finishes with a tagged production release. The process is evidence-led: if a change cannot show what was changed, why it was changed, who approved it and how it was tested, it is not ready for production.

Power BI Change Process - Swimlane View



Control notes: rejected changes return to feature branch; emergency hotfixes require retrospective documentation within one working day.

Figure 3. Swimlane process for Power BI change control and release promotion.

1. **1. Intake:** Change request is logged with business rationale, affected dashboard, KPI impact and target release window.
2. **2. Impact assessment:** Analyst confirms whether the change affects visuals only, DAX logic, model relationships, data sources, RLS, refresh schedules or business definitions.
3. **3. Development:** Change is developed locally using PBIP/project structure and committed to a feature branch.
4. **4. Peer review:** Pull request is reviewed for logic, naming, performance, documentation and downstream impact.
5. **5. CI validation:** Automated checks run before merge or deployment to shared workspace.
6. **6. DEV deployment:** Change is deployed to DEV workspace for integrated validation.
7. **7. TEST/UAT:** Refresh, reconciliation, performance and business acceptance checks are completed.
8. **8. Production release:** Approved release is promoted to production, tagged and monitored during the release window.
9. **9. Post-release review:** Any incidents, defects or user feedback are logged against the release.

9. Release Gates and Quality Controls

Release gates are used to prevent avoidable production defects. A gate is passed only when the required checks are completed and the evidence is stored with the release note.

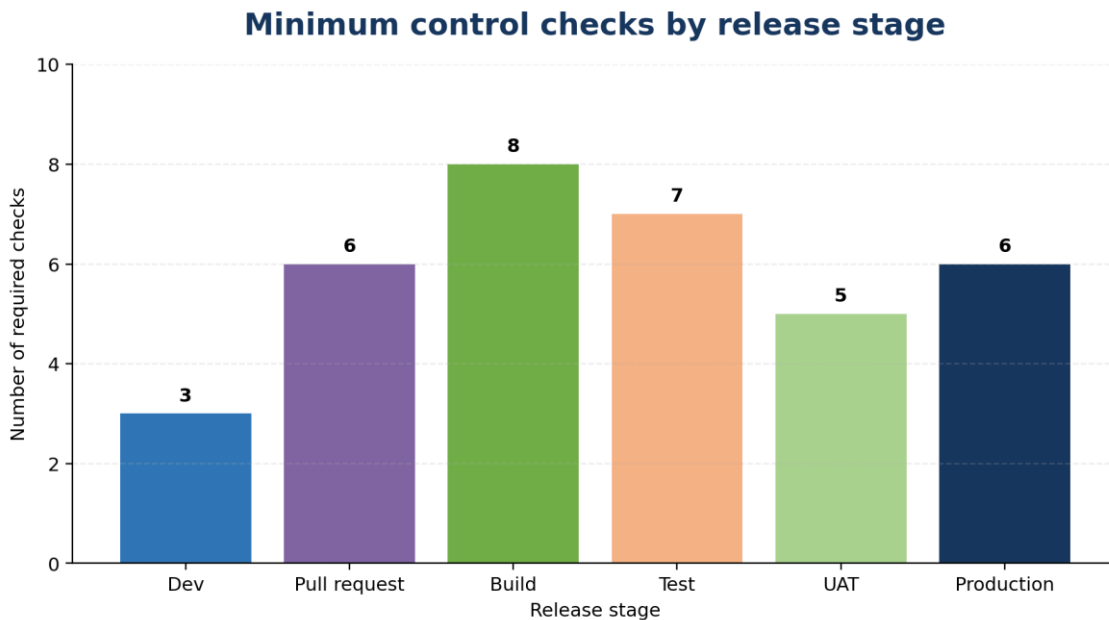


Figure 4. Minimum control checks expected at each stage of the Power BI release path.

Gate	Pass criteria	Evidence required
Gate 0 - Intake complete	Change request contains purpose, owner, affected report/model, KPI impact and expected benefit.	Change request log
Gate 1 - Development ready	Data source impact, RLS impact and downstream consumers are assessed before build.	Impact assessment note
Gate 2 - Pull request passed	Peer review completed; no unresolved comments; documentation updated.	PR approval
Gate 3 - CI validation passed	Automated checks pass for structure, metadata, naming, model quality and basic report standards.	CI run evidence
Gate 4 - TEST refresh passed	Dataset refresh succeeds; row counts and key KPI totals are reconciled against agreed source extracts.	Refresh/reconciliation log
Gate 5 - UAT approved	Business owner confirms KPI interpretation, dashboard layout and filter behaviour.	UAT sign-off
Gate 6 - Production release approved	Release note, version tag, rollback option and deployment window are confirmed.	Release approval record

10. Testing Framework

Testing is split into technical checks, data checks, user checks and operational checks. The level of testing depends on change type: a visual-only change requires lighter testing than a change to DAX measures, model relationships, RLS or data source logic.

Test area	Validation activity	When required
PBIP/project structure	Confirm required report/model folders exist and no generated cache files or credentials are committed.	Every PR
DAX measure review	Check naming, formatting, unused measures, ambiguous filter context and KPI definition alignment.	Measure changes
Power Query review	Check source parameters, applied steps, privacy level, query folding where applicable and refresh impact.	Data prep changes
Model relationship review	Validate cardinality, cross-filter direction, inactive relationships and role-playing dates.	Model changes
Refresh validation	Confirm scheduled refresh success, duration, gateway status and data currency.	Every release
KPI reconciliation	Compare key totals against SQL extracts or agreed baseline reports.	Every release
RLS/security test	Validate user role visibility using test accounts or role simulation.	Security-impacting releases
Performance test	Check page load, visual query duration, model size and high-cost visuals.	Material report/model changes
UAT scenario test	Validate agreed business scenarios such as site drill-through, date filter, PCN funnel movement or fault ageing.	Every production release
Post-release smoke test	Confirm app access, refresh schedule, top-level KPI cards and key navigation links after deployment.	Every production release

11. Deployment Rules and Configuration Management

Environment-specific settings should be handled through deployment rules, parameters or documented deployment configuration rather than hard-coded into report logic. This prevents a report from accidentally pointing production users at development data or test credentials.

Deployment Path and Rollback Control

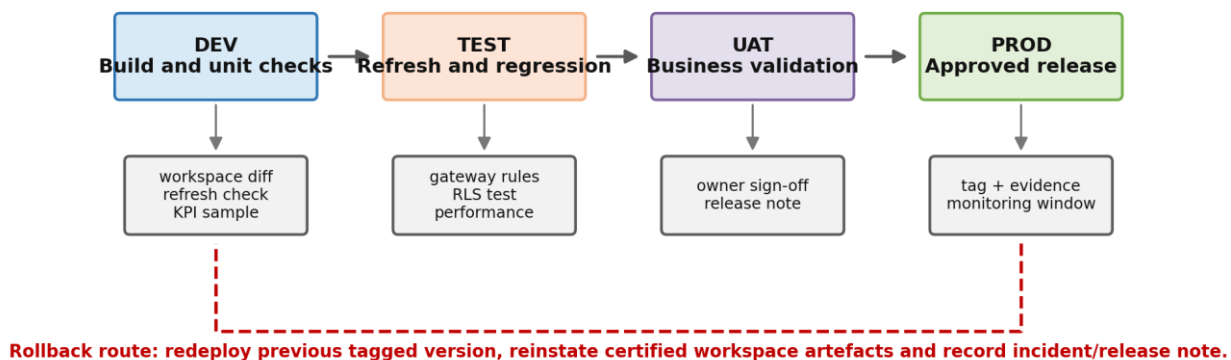


Figure 5. Deployment flow with release gates, approval points and rollback route.

Configuration item	Control requirement
Data source parameters	Use parameters for server, database, schema or workspace-specific endpoints where practical.
Deployment rules	Use deployment-stage rules to switch data source or parameter values between DEV, TEST and PROD.
Gateway mapping	Document which gateway and credential owner supports each environment.

Configuration item	Control requirement
Refresh schedules	Keep refresh cadence aligned to operational need and source system readiness.
Secrets and credentials	Do not store secrets in PBIP files, scripts or release notes.
Semantic model endorsement	Production shared models should be labelled and documented so report builders use the certified source.
Rollback configuration	Previous release tag, previous deployment package and workspace state must be known before promotion.

12. Roles and Responsibilities

A RACI model prevents ambiguity during report change, release approval and incident response. The matrix below should be adjusted as the analytics operating model matures.

Activity	Analytics developer	Analytics lead	Workspace admin	Operations owner	Senior stakeholder
Change request intake	R	A	C	C	I
Impact assessment	R	A	C	C	I
PBIP development	R	A	I	C	I
Pull request review	C	A/R	I	C	I
CI validation	R	A	C	I	I
Deployment to DEV/TEST	R	A	R	I	I
UAT sign-off	C	A	I	R	I
Production deployment approval	C	A	R	C	I
Production incident response	R	A	R	C	I
Release communication	C	A/R	I	C	I

R = Responsible, A = Accountable, C = Consulted, I = Informed.

13. Naming Standards and Folder Structure

Asset type	Standard pattern	Example
Reports	pbi-rpt-[domain]-[purpose]	pbi-rpt-operations-performance
Semantic models	pbi-sm-[domain]-[grain]	pbi-sm-pcn-daily-site
Workspaces	PBI-[ENV]-[Domain]	PBI-PROD-Operations
Branches	feature/PBI-####-[description]	feature/PBI-1042-add-site-filter
Tags	vYYYY.MM.release	v2026.06.1
Measures	[Business noun] [Metric]	PCN Issued Count
Pages	[Process] - [View]	Fault Control - Ageing
Parameters	p_[Environment]_[Setting]	p_Prod_Server
Release notes	release-vYYYY.MM.release.md	release-v2026.06.1.md

Commit Message Examples

- `feat(model): add site dimension relationship for operations dashboard` - New feature or model capability.
- `fix(dax): correct PCN conversion denominator` - Defect fix in a measure or calculation.
- `docs(kpi): update exemption metric definition` - Documentation-only change.
- `refactor(query): simplify incident status staging query` - Technical improvement without business logic change.
- `hotfix(prod): restore fault-control refresh parameter` - Urgent production fix.

14. Release Calendar and Operational Cadence

The release cadence is designed to protect daily operational reporting while still allowing urgent fixes when production reporting is materially affected.

Cadence	Timing	Purpose
Weekly change review	30 minutes	Review new requests, classify impact,

Cadence	Timing	Purpose
		assign owner and target release.
Development window	Monday to Wednesday	Build and peer review standard changes.
TEST/UAT window	Thursday	Validate refresh, KPI totals, visual behaviour and business acceptance.
Production release window	Friday morning or agreed low-impact period	Deploy approved release, tag version and monitor dashboard health.
Post-release check	Same day	Confirm refresh, access, key KPI cards, subscriptions and app navigation.
Monthly governance review	Monthly	Review defects, release volume, refresh failures, model performance and backlog trends.

15. Implementation Roadmap

Phase	Timeframe	Key activities
Phase 1 - Stabilise	Week 1-2	Inventory reports, identify production dashboards, confirm owners, define workspace permissions and freeze direct production edits.
Phase 2 - Source control baseline	Week 3-4	Convert priority reports to PBIP/project structure, create repository folders, commit baseline and add README/release note templates.
Phase 3 - Release gates	Week 5-6	Introduce PR template, test checklist, RACI, release log and basic deployment evidence pack.
Phase 4 - Environment promotion	Week 7-8	Set up DEV/TEST/PROD workspace flow, document deployment rules and run first controlled release.
Phase 5 - Automation maturity	Week 9-12	Add CI checks, automated validation scripts, deployment pipeline automation and release metrics dashboard.
Phase 6 - Continuous improvement	Ongoing	Review release defects, refresh failures, dashboard performance and stakeholder feedback monthly.

15.1 Key Risks and Mitigations

Risk	Scenario	Mitigation
Uncontrolled production edit	A report is changed directly in the production workspace without peer review.	Restrict publisher access; monitor workspace activity; require release tag for all production changes.
Incorrect KPI definition	A measure change changes the operational meaning of a KPI such as issued PCNs, exemption counts or fault ageing.	Require KPI dictionary update and business owner sign-off for definition-impacting changes.
Data source mismatch	A test or production report points to the wrong database, gateway or parameter value.	Use deployment rules, parameter checks and post-deployment source validation.
Refresh failure after release	A model refresh fails because of changed source schema or gateway credentials.	Run TEST refresh before production; maintain rollback version and refresh ownership.
Security leakage	RLS or workspace access allows unauthorised visibility of restricted operational views.	Run RLS validation and workspace membership review before release.
Over-complex release process	Controls become too heavy and analysts bypass the process.	Use risk-based gates: visual-only changes follow a lighter path; data/model/security changes follow full review.

15.2 Adoption Success Measures

- 100% of production Power BI changes linked to a release note and version tag.
- No recurring direct edits in the production workspace outside the emergency hotfix path.
- Reduction in production dashboard defects caused by undocumented measure or filter changes.

- Clear ownership for all certified semantic models and operational dashboards.
- Faster recovery from reporting issues because previous stable versions are available and documented.

16. Appendices

Appendix A - Pull Request Checklist

- ☐ Work item or change request linked.
- ☐ Impact assessment completed for KPI definitions, data source, RLS, refresh and downstream users.
- ☐ Report/model saved in the agreed repository folder structure.
- ☐ No credentials, extracts, sample personal data or local cache files committed.
- ☐ DAX measures are formatted, named consistently and aligned to KPI dictionary definitions.
- ☐ Power Query changes are reviewed for refresh and query folding impact where applicable.
- ☐ Report pages follow layout standards and do not introduce duplicate or conflicting KPI definitions.
- ☐ Refresh test completed in DEV or TEST workspace.
- ☐ Key KPI totals reconciled against source query or approved baseline.
- ☐ Business owner approval captured for any visible KPI, filter, layout or definition change.
- ☐ Release note and rollback position completed before production deployment.

Appendix B - Release Note Template

Field	Description
Release version	vYYYY.MM.release
Release date	DD Month YYYY
Dashboard / model impacted	Name of report, semantic model and workspace
Business purpose	Why the change is required
Change summary	Visual, DAX, model, Power Query, data source, security or documentation change
Testing completed	Refresh, reconciliation, UAT, RLS, performance and smoke test results
Approval	Named approver and date
Rollback plan	Previous version tag and recovery steps
Known limitations	Any known issues or follow-up actions

Appendix C - Release Metrics

Metric	Definition	Target / usage
Deployment success rate	Successful production deployments / total deployments.	Target: >= 95%
Rollback rate	Deployments requiring rollback / total production deployments.	Target: <= 5%
Defect leakage	Post-production defects linked to released dashboard changes.	Target: downward trend
Refresh reliability	Successful scheduled refreshes / total scheduled refreshes.	Target: >= 98%
Cycle time	Elapsed time from approved intake to production deployment.	Target: reduce without bypassing controls
PR review compliance	Pull requests with completed peer review / total merged PRs.	Target: 100%
UAT evidence compliance	Releases with captured UAT sign-off / total production releases.	Target: 100%

Appendix D - Reference Points

The framework aligns with current Microsoft Power BI and Fabric application lifecycle management patterns, adapted into a practical operating model for an operational analytics function.

- [Microsoft Learn - Power BI Desktop projects \(PBIP\)](#)

- [Microsoft Learn - Git integration with Power BI Desktop projects](#)
- [Microsoft Learn - Azure DevOps integration with Power BI Desktop projects](#)
- [Microsoft Learn - Azure DevOps build pipeline integration with Power BI Desktop projects](#)
- [Microsoft Learn - Microsoft Fabric Git integration](#)
- [Microsoft Learn - Fabric deployment pipelines](#)
- [Microsoft Learn - Fabric deployment rules](#)
- [Microsoft Learn - Deploy Power BI projects using fabric-cicd](#)

Appendix E - Practical Operating Notes

Operating note	Practical application
Keep one certified semantic model per operational domain	Where practical, reports should connect to a certified model rather than duplicating measure logic across multiple PBIX files.
Treat KPI definition changes as business changes	A visual change can be reviewed by analytics; a metric logic change needs business owner review because it changes operational interpretation.
Use hotfixes sparingly	Hotfixes are for material production issues only. Cosmetic changes should wait for the normal release window.
Retain release evidence	The release note, screenshots, reconciliation results and approval record should be stored with the version tag or release folder.
Review the framework monthly	Controls should be adjusted based on release defects, refresh reliability, stakeholder feedback and dashboard usage.